바이브 코딩 7일 완성 — 클로드와 함께하는 첫 개발

DAY 7 / 7

MCP로 클로드 코드의 한계 넘어서기

노션, 깃허브, 데이터베이스까지 연결해 나만의 서비스를 세상에 내놓는 마지막 하루

교육 시간 09:00 - 18:00 (8시간)

대상 마케팅 실무자·마케터 (바이브 코딩 입문)

학습 목표

✓ MCP(Model Context Protocol)의 개념을 이해하고 클로드 코드와 연결할 수 있다.
✓ 원격 MCP와 로컬 MCP 서버를 구분하고 필요에 맞게 설치할 수 있다.
✓ Playwright MCP로 웹 애플리케이션이 제대로 작동하는지 자동으로 테스트할 수 있다.
✓ GitHub MCP를 통해 프로젝트를 버전 관리하고 저장소에 업로드할 수 있다.
✓ Vercel과 Supabase를 활용해 데이터베이스가 연결된 실제 서비스를 배포하고, 공개 전에 개인정보가 노출되지 않도록 보안 설정을 점검할 수 있다.

오늘의 시간표 (총 8시간)

시간	구성	내용
09:00 - 09:40	1교시 · 개념	MCP란 무엇인가 — 클로드 코드의 한계를 넘는 연결 장치
09:40 - 09:50	쉬는 시간	
09:50 - 10:35	2교시 · 개념+실습	클로드 코드에 첫 MCP 서버 연결하기 — 노션 MCP
10:35 - 10:45	쉬는 시간	
10:45 - 11:45	3교시 · 개념+실습	로컬 MCP와 원격 MCP 구분, Node.js 설치, Playwright MCP 연결
11:45 - 11:55	쉬는 시간	
11:55 - 12:30	4교시 · 개념+실습	오후 준비 — 깃허브·Supabase 가입과 MCP 연결
12:30 - 13:30	점심시간	
13:30 - 15:20	5교시 · 실습 프로젝트	랜딩페이지 리드 수집 앱 제작 → 자동 테스트 → 깃허브 업로드 → Vercel 배포
15:20 - 15:30	쉬는 시간	
15:30 - 17:10	6교시 · 실습 프로젝트	Supabase 데이터베이스 연동, 리드 현황, 공개 전 보안 설정으로 진짜 서비스 완성
17:10 - 18:00	7교시 · 총정리	7일 과정 총정리, 확인 문제, 수료식

1교시 · MCP란 무엇인가 — 클로드 코드의 한계를 넘는 연결 장치 09:00 - 09:40

지난 6일 동안 우리는 클로드 코드로 프로그램을 만들고, API로 외부 AI 모델을 가져와 쓰고, 서브에이전트에게 역할을 나눠 맡기고, 공공데이터로 상권 조회 웹까지 만들어 봤습니다. 그런데 한 가지 벽은 늘 남아 있었습니다. 클로드 코드는 내 컴퓨터 안의 파일과 코드는 자유롭게 다루지만, 노션 문서를 대신 써 주거나 깃허브에 직접 코드를 올리거나 실제 데이터베이스를 조회하는 일은 하지 못했습니다. 마지막 날인 오늘, 바로 이 벽을 허무는 방법을 배웁니다.

레고 블록의 비유로 이해하는 MCP

레고 기본 세트만으로도 자동차나 집 정도는 만들 수 있습니다. 하지만 움직이는 바퀴, 반짝이는 조명, 진짜 작동하는 모터 같은 특별한 부품을 더하면 상상했던 거의 모든 것을 만들 수 있습니다. MCP(Model Context Protocol, 모델 컨텍스트 프로토콜)는 클로드 코드라는 기본 세트에 이런 특별한 부품을 갈아 끼울 수 있게 해 주는 표준 연결 장치입니다. 노션, 깃허브, 데이터베이스, 브라우저 제어 도구 등이 모두 이 연결 장치를 통해 클로드 코드에 부품처럼 결합됩니다.

> 📘 **용어 정리 · MCP(Model Context Protocol)**
>
> AI 모델이 외부 데이터, 도구, 서비스와 연결될 때 사용하는 표준화된 통신 규약입니다.
>
> 클로드를 만든 앤트로픽(Anthropic)이 2024년 11월에 처음 제안했고, 지금은 특정 회사를 넘어 전 세계 AI 개발자들이 함께 쓰는 업계 표준이 되었습니다.
>
> 클로드 코드는 MCP를 사용할 수 있는 기능(MCP 클라이언트)을 내장하고 있어서, 별도의 프로그램 설치 없이 명령어 한 줄로 새로운 MCP 서버를 연결할 수 있습니다.

MCP의 'C' — 컨텍스트(Context)란 무엇인가

MCP라는 이름 가운데 글자 C는 컨텍스트(Context)입니다. 컨텍스트는 **AI가 지금 이 순간 보면서 판단할 수 있는 정보 전체**를 말합니다. 내가 입력한 대화, 클로드 코드가 읽은 파일, CLAUDE.md의 규칙, 명령을 실행한 결과가 모두 컨텍스트에 들어갑니다. 반대로 컨텍스트에 없는 정보는 아무리 중요해도 AI가 알 수 없습니다. 클로드 코드가 내 노션 문서를 모르는 이유는 똑똑하지 않아서가 아니라, 그 문서가 컨텍스트에 들어온 적이 없기 때문입니다.

책상 위에서 일하는 비서의 비유

클로드 코드를 아주 유능하지만 **자기 책상 위에 놓인 자료만 보고 일하는 비서**라고 생각해 봅시다. 비서는 책상 위 서류를 읽고, 메모를 보며, 그 자리에서 결과물을 만듭니다. 책상 밖 캐비닛에 있는 자료는 누군가 꺼내 와서 책상에 올려 주기 전까지 모릅니다.

비유	클로드 코드에서는	지금까지 배운 것과 연결하면
책상 위에 올라온 서류	컨텍스트	대화 내용, 읽은 파일, 실행 결과
책상의 크기	컨텍스트 창(context window) — 한 번에 담을 수 있는 양의 한계	대화가 길어지면 책상이 가득 참
책상에 늘 붙어 있는 메모	CLAUDE.md	세션을 새로 시작해도 처음부터 올라와 있는 규칙 (3일차)
책상 깨끗이 치우기	`/clear`	주제가 바뀌면 이전 서류를 모두 내림 (5일차)
서류 더미를 요약 메모 한 장으로 바꾸기	`/compact`	중요한 결론만 남기고 자리를 비움 (4일차)
옆 책상의 다른 비서에게 맡기기	서브에이전트	자기 책상에서 일하고 결과 요약만 건넴 (6일차)
캐비닛·다른 부서와 연결된 서류 배달 창구	**MCP**	노션·깃허브·데이터베이스의 자료를 필요할 때 책상 위로 가져오고, 결과를 다시 보냄 (오늘)

즉 MCP는 바깥 서비스의 정보를 AI의 컨텍스트로 가져오고, AI가 내린 결정을 다시 바깥 서비스에 전달하는 **표준 배달 창구**입니다. 창구 모양이 표준으로 정해져 있어서, 노션이든 깃허브든 같은 방식으로 연결할 수 있습니다. 이름 그대로 '모델(Model)에게 컨텍스트(Context)를 전달하는 규약(Protocol)'입니다.

> ✋ **여기서 잠깐 · 책상이 넓다고 다 좋은 것은 아닙니다**
>
> 최신 모델은 책상이 매우 넓지만, 서류가 산더미처럼 쌓이면 사람처럼 AI도 앞쪽에 있던 지시를 놓치거나 오래된 내용과 새 내용을 섞기 시작합니다. 대화가 길어진 뒤 답이 엉뚱해졌다면 능력이 떨어진 것이 아니라 책상이 어지러워진 것일 가능성이 큽니다. 작업 하나가 끝나면 `/clear`로 치우고, 계속 이어 가야 하면 `/compact`로 정리하는 습관이 결과 품질을 지킵니다.
>
> MCP를 여러 개 연결하면 각 MCP가 제공하는 **도구 설명서**도 책상 자리를 차지할 수 있습니다. 오늘 실습에 필요한 MCP만 연결하고, 다 쓴 MCP는 `claude mcp remove 이름`으로 정리합니다.

🛠 직접 확인 · 내 책상 위에는 무엇이 올라와 있을까

2교시에 노션 MCP를 연결한 뒤, 클로드 코드 입력창에 `/context` 를 입력해 보세요. 지금 컨텍스트를 무엇이 얼마나 차지하고 있는지(시스템 설정, CLAUDE.md 같은 메모 파일, MCP 도구, 대화 내용, 남은 공간)가 칸으로 나뉘어 보입니다. 노션에 리서치를 저장한 뒤 한 번 더 입력하면 대화가 늘어난 만큼 칸이 차오른 것을 확인할 수 있습니다.

API와는 무엇이 다를까

6일 차에 '포토애드'를 만들면서 오픈라우터의 API 키로 AI 모델을 가져와 이미지 인식과 텍스트 생성에 활용했습니다. API는 다른 프로그램의 기능을 잠깐 빌려 쓰는 창구입니다. 하나의 요청을 보내면 하나의 결과를 돌려받는, 말하자면 일회성 심부름과 비슷합니다. 반면 MCP는 여러 API와 파일, 데이터베이스를 한꺼번에 통합해 연결하는 프로토콜입니다.

구분	API	MCP
역할	특정 기능 하나를 요청하고 응답받는 창구	여러 도구·데이터·서비스를 통합해 AI에 연결하는 규약
비유	필요할 때마다 빌리는 물건 하나	여러 물건을 한 번에 꽂아 쓰는 멀티탭
누가 쓰나	우리가 만든 앱이 코드 안에서 호출 (6일 차 포토애드·상권 조회 웹)	클로드 코드(AI)가 작업하면서 필요한 도구를 스스로 골라 사용
할 수 있는 일	정해진 기능 하나를 요청하고 결과를 받음	노션 문서, 깃허브 저장소, 데이터베이스 같은 외부 서비스를 직접 조회·수정

정리하면, 클로드 코드는 원래 내 컴퓨터 안의 파일까지만 다룰 수 있었습니다. 노션 문서나 깃허브 저장소, 클라우드 데이터베이스처럼 바깥에 있는 서비스는 MCP로 연결해야 직접 다룰 수 있습니다. 오늘 하루 동안 노션 문서 자동 작성, 웹 앱 자동 테스트, 깃허브 자동 업로드, 데이터베이스 자동 연동까지 경험합니다.

✋ 여기서 잠깐 · 클라이언트-서버 모델

클라이언트-서버 모델은 한쪽이 정보를 요청하고 다른 한쪽이 응답하는 구조입니다. 요청하는 쪽이 클라이언트, 서비스를 제공하는 쪽이 서버입니다. 클로드의 MCP도 이 구조를 따릅니다. 클로드가 클라이언트로서 '이 작업을 해 줘'라고 요청하면, 각 MCP 서버가 실제 작업을 처리하고 결과만 돌려줍니다.

새 기능이 필요하면 MCP 서버 하나만 추가하면 되므로 확장이 쉽고, 실제 실행은 각 서버가 전담하므로 안정성이 높습니다. 다만 MCP 서버가 외부 파일이나 API에 접근하는 만큼 보안 인증과 접근 권한 설정은 꼼꼼히 확인해야 합니다.

2교시 · 클로드 코드에 첫 MCP 서버 연결하기 — 노션 MCP 09:50 - 10:35

오늘의 첫 실습 파트너는 문서 작성·협업 도구인 노션(Notion)입니다. 클로드 코드가 노션과 대화할 수 있게 되면, 부탁한 내용을 스스로 조사해 노션 페이지로 정리해 주는 '리서치 비서'를 갖게 됩니다.

① 노션 가입하고 실습 폴더 준비하기

STEP 1 노션 가입

`https://www.notion.com` 접속 → 우측 상단 [Notion 무료로 사용하기] → 이메일 또는 다른 서비스 계정으로 가입합니다.

STEP 2 워크스페이스 생성

안내에 따라 워크스페이스를 하나 만들면 노션을 쓸 준비가 끝납니다.

STEP 3 실습 폴더 만들기

'VibeCoding' 폴더 안에 오늘 실습용 'Study-07' 폴더를 만들고, VS Code에서 [File] → [Open Folder]로 엽니다. 신뢰 확인 창에서 [예, 작성자를 신뢰합니다]를 누르고, **Ctrl+`**로 통합 터미널을 열어 둡니다. 아직 클로드 코드는 실행하지 않습니다.

> 📘 **오늘은 VS Code에서 실습합니다**
> 4일차 부록 「VS Code와 클로드 코드 함께 사용하기」에서 익힌 대로, 실습 폴더를 VS Code로 열고 화면 아래 **통합 터미널**에서 `claude` 를 실행합니다. 한 화면에서 세 가지를 함께 봅니다.
>
> - **왼쪽 탐색기** — 클로드 코드가 만든 `lead-capture.html` , `test-checklist.md` 같은 파일이 바로 나타납니다. 점(.)으로 시작하는 설정 파일도 숨김 설정 없이 보입니다.
> - **가운데 편집기** — 만든 파일을 직접 열어 고치고 **Ctrl+S**로 저장합니다. `.md` 문서는 **Ctrl+Shift+V**를 누르면 제목·표가 정리된 미리보기로 읽을 수 있습니다.
> - **아래 통합 터미널** — 클로드 코드와 대화하고, 서버 주소 같은 실행 결과를 봅니다. 터미널에 나온 주소는 **Ctrl+클릭**하면 브라우저로 열립니다.
>
> 클로드 코드가 파일을 고치면 VS Code의 **비교 화면(diff)**으로 바뀐 부분을 보여 줍니다. 확장 프로그램의 대화창 대신 통합 터미널을 쓰는 이유는, 수업에서 쓰는 모든 명령어(`claude mcp add` , `/mcp` , `/context` 등)를 그대로 쓸 수 있기 때문입니다.
>
> **터미널을 두 개 씁니다.** MCP 설치 명령(`claude mcp add …`)은 클로드 코드 밖, 일반 터미널에서 실행해야 합니다. 통합 터미널 오른쪽 위의 **[+]** 버튼으로 터미널을 하나 더 열어, **터미널 1은 클로드 코드 대화용**, **터미널 2는 설치·확인 명령용**으로 나눠 쓰면 클로드 코드를 껐다 켤 일이 줄어듭니다. 터미널 목록에서 이름을 눌러 오가거나, 분할 아이콘으로 나란히 볼 수 있습니다.

② MCP 서버 설치 명령어 찾아서 실행하기

MCP 서버 추가는 언제나 같은 흐름입니다. 터미널에서 `claude mcp add`로 서버를 설치하고, 외부 서비스라면 계정 인증을 거친 뒤, 클로드 코드 안에서 프롬프트로 기능을 사용합니다. 설치 명령어는 클로드 코드 공식 문서에서 확인합니다.

STEP 1 공식 문서 접속

`https://code.claude.com/docs/en/mcp`에 접속해 'Installing MCP servers' 항목을 찾습니다.

STEP 2 노션 설치 명령어 복사

'Option 1: Add a remote HTTP server' 아래 회색 명령어 상자에서 `# Real example: Connect to Notion` 줄 바로 아래의 명령어를 복사합니다.

STEP 3 터미널에서 실행

VS Code 통합 터미널에 붙여 넣고 Enter를 누릅니다. 'Added HTTP MCP server'로 시작하는 메시지가 보이면 성공입니다.

노션 MCP 서버 설치 명령어

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

③ 클로드 코드 실행 후 인증으로 연결 완성하기

STEP 1 클로드 코드 실행

같은 통합 터미널(터미널 1)에 `claude`를 입력해 실행합니다. 이후 설치 명령은 [+]로 연 터미널 2에서 실행합니다.

STEP 2 MCP 목록 확인

`/mcp`를 입력하면 노션 MCP가 등록되어 있지만 아직 로그인 전이라 연결되지 않은 상태로 표시됩니다.

STEP 3 인증 시작

노션 항목을 선택하고 Enter → 'Authenticate'를 선택하고 다시 Enter를 누릅니다.

STEP 4 브라우저에서 승인

새 브라우저 창에서 '이 URL을 알고 신뢰합니다.'에 체크하고 [계속하기]를 누르면 'Authentication Successful' 메시지가 나타납니다.

STEP 5 연결 재확인

클로드 코드로 돌아와 다시 `/mcp`를 입력하면 노션 MCP가 연결된 상태로 표시됩니다. Enter로 상세 정보를 열면 클로드가 쓸 수 있는 작업 도구 목록이 보입니다. 확인 후 Esc로 닫습니다.

④ 실전 — 노션 MCP로 자동 리서치 노트 만들기

오늘 만들 것은 방문자의 이메일을 모으는 '랜딩페이지 리드 수집 앱'입니다. 개발에 앞서 참고할 경쟁사 사례를 검색해 노션 문서로 정리해 달라고 요청합니다.

클로드 코드 프롬프트 입력 예시

```
국내외 스타트업들이 쓰는 랜딩페이지 리드 수집 폼 사례를 3~4개 검색해서,
각각 어떤 입력 항목을 받는지, 디자인과 문구는 어떤 특징이 있는지
정리한 다음 노션에 새 페이지로 저장해 줘.
```

권한 확인 창이 뜨면 'Yes'를 눌러 진행합니다. 잠시 뒤 노션에 정리해 저장했다는 메시지가 나옵니다. 노션에 접속해 새 페이지를 확인해 보세요. 대화 몇 줄만으로 리서치와 문서화가 끝났다는 것이 MCP의 힘입니다. 오후에 리드 수집 앱을 만들 때 이 내용을 참고할 수 있습니다.

3교시 · 로컬 MCP와 원격 MCP — Playwright MCP 10:45 - 11:45

원격 MCP와 로컬 MCP, 무엇이 다를까

방금 연결한 노션 MCP는 '원격(remote) MCP'였습니다. 인터넷 저편 노션 서버에 준비된 MCP에 접속만 하므로 명령어 한 줄과 로그인만으로 연결됐습니다. 반면 '로컬(local) MCP'는 중개 프로그램을 내 컴퓨터 안에서 직접 실행합니다.

구분	원격 MCP	로컬 MCP
실행 위치	인터넷 저편 클라우드 서버	내 컴퓨터 안
연결 방법	이미 준비된 URL 주소로 접속	실행할 명령어를 직접 지정
설정 난이도	명령어 한 줄과 로그인	Node.js 같은 실행 도구가 필요
오늘 쓰는 것	노션, 깃허브, Supabase	Playwright

> ✋ **여기서 잠깐 · 인증(authentication)이란?**
> 외부 서비스와 MCP를 연결할 때는 접속하는 사람이 정말 허가된 사용자인지 확인하는 절차가 필요합니다. 노션에서 브라우저로 로그인해 승인한 것이 바로 인증입니다. 인증을 거치면 클로드 코드는 **내 계정이 허락한 범위 안에서만** 그 서비스를 다룰 수 있습니다.

① Node.js 설치 확인

로컬 MCP를 실행하는 데 필요한 `npx`는 Node.js에 포함된 기능입니다. 먼저 설치되어 있는지 확인합니다.

설치 확인 명령어

```
node -v
```

버전이 표시되면 이미 설치된 것이므로 아래 설치 절차는 건너뜁니다.

STEP 1 공식 사이트 접속

`https://nodejs.org/ko` → [다운로드]에서 Windows 설치 프로그램(.msi, LTS 버전 권장)을 내려받습니다.

STEP 2 설치

설치 파일을 실행하고 모든 옵션을 기본값으로 둔 채 [Next] → [Install] → [Finish]를 누릅니다. 이어서 뜨는 추가 도구 설치용 창은 닫아도 됩니다.

STEP 3 VS Code를 다시 열어 확인

VS Code를 **완전히 닫았다가 다시 열어야** 새로 설치한 Node.js를 인식합니다. 클로드 코드 대화가 진행 중이면 `/exit`로 먼저 종료하고 닫습니다. 다시 연 VS Code의 통합 터미널에서 `node -v`와 `npm -v`를 입력해 버전이 표시되는지 확인합니다. 클로드 코드는 `claude --continue`로 실행하면 하던 대화를 이어 갈 수 있습니다.

② Playwright MCP — 브라우저를 직접 조작하는 도구

버튼이 제대로 작동하는지, 입력값이 올바르게 저장되는지를 매번 손으로 눌러 확인하는 일은 시간이 걸리고 실수하기 쉽습니다. Playwright MCP를 연결하면 클로드 코드가 실제 브라우저를 열어 클릭하고 입력하면서 확인 작업을 대신합니다. 오후에 만들 앱을 자동으로 테스트하는 데 씁니다.

Playwright MCP 설치 명령어 (Windows)

```
claude mcp add playwright -- cmd /c npx -y @playwright/mcp@latest
```

> 📘 **용어 정리 · 명령어 읽는 법**
>
> **claude mcp add playwright** : 'playwright'라는 이름으로 MCP 서버를 추가합니다.
>
> -- 뒤의 부분 : 이 MCP 서버를 내 컴퓨터에서 실행하는 명령입니다. `npx` 가 Playwright 팀이 배포한 프로그램을 내려받아 실행하고, `cmd /c` 는 Windows에서 이 명령을 실행하게 해 줍니다. macOS에서는 `cmd /c` 를 빼고 입력합니다.
>
> 명령어를 외울 필요는 없습니다. 공식 문서나 각 MCP의 안내 페이지에서 복사해 쓰면 됩니다.

STEP 1 설치

VS Code의 **터미널 2**(클로드 코드가 실행되지 않은 터미널)에 위 명령어를 입력합니다.

STEP 2 설치 확인

`claude mcp list`로 'playwright'가 목록에 있고 연결된 상태인지 확인합니다.

STEP 3 처음 써 보기

터미널 1에서 클로드 코드를 `/exit` 후 `claude`로 다시 실행해 새 MCP를 불러오고, 아래처럼 요청합니다. 브라우저 창이 저절로 열리고 움직이는 모습을 지켜봅니다.

클로드 코드 프롬프트 입력 예시

```
Playwright MCP로 브라우저를 열어서 우리 회사(또는 자주 가는 브랜드) 홈페이지에 접속하고,
첫 화면 스크린샷을 찍은 다음 메뉴에 어떤 항목이 있는지 정리해 줘.
```

✋ **여기서 잠깐 · 원하는 MCP 서버를 찾는 방법**

MCP는 누구나 만들어 배포할 수 있어 수천 가지가 넘습니다. 클로드 코드 공식 문서에도 몇 가지가 소개되어 있고, 원하는 기능을 검색하거나 MCP 서버를 카테고리별로 모아 둔 플랫폼에서 찾을 수 있습니다. MCP 서버는 내 계정과 파일에 접근하므로 **공식 회사나 출처가 확실한 서버만** 설치합니다.

4교시 · 오후 준비 — 깃허브·Supabase 가입과 MCP 연결 11:55 - 12:30

오후에는 앱을 만들고 → 테스트하고 → 깃허브에 보관하고 → 배포하고 → 데이터베이스까지 연결합니다. 가입과 연결은 시간이 들고 사람마다 막히는 지점이 달라서, 점심 전에 미리 끝내 두면 오후에는 만드는 일에만 집중할 수 있습니다.

① 깃허브(GitHub) 가입과 MCP 연결

깃허브는 전 세계 개발자가 쓰는 코드 보관·버전 관리 플랫폼입니다. 깃허브 MCP를 연결하면 클로드 코드가 저장소 만들기와 업로드를 대신 처리합니다.

STEP 1 깃허브 가입

`https://www.github.com` → [Sign up]으로 계정을 만듭니다. 계정이 있다면 로그인합니다.

STEP 2 토큰 메뉴 이동

프로필 → [Settings] → 왼쪽 아래 [Developer settings] → [Personal access tokens] → [Tokens (classic)] → [Generate new token (classic)]을 선택합니다.

STEP 3 토큰 정보 입력

'Note'에 용도(예: `vibecoding-day7`), 'Expiration'은 **'7 days'**로 지정하고, 'Select scopes'에서 **repo 하나만** 체크한 뒤 [Generate token]을 누릅니다.

STEP 4 토큰 복사

이 화면을 벗어나면 다시 볼 수 없으므로 바로 복사해 메모장에 잠시 둡니다.

깃허브 MCP 설치 명령어

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ -H "Authorizati
on: Bearer <깃허브 토큰>"
```

STEP 5 설치와 연결 확인

`<깃허브 토큰>` 자리에 복사한 토큰을 붙여 넣어 실행하고, `claude mcp list`로 github가 연결된 상태인지 확인합니다. 확인이 끝나면 메모장에 둔 토큰은 지웁니다.

✋ **여기서 잠깐 · 토큰은 '필요한 만큼, 짧게'**

토큰은 내 깃허브 계정의 열쇠입니다. 그래서 권한은 저장소를 다루는 `repo` 하나만, 기간은 실습에 필요한 7일만 줍니다. 과정이 끝나면 같은 [Tokens (classic)] 화면에서 이 토큰을 삭제합니다. 토큰이 노출됐다고 의심되면 바로 삭제하고 새로 만듭니다.

설치 명령어에 넣은 토큰은 클로드 코드 설정 파일(사용자 폴더의 `.claude.json`)에 그대로 저장됩니다. 이 파일을 다른 사람에게 보내거나 화면 공유로 열어 보이지 않습니다. 토큰을 삭제할 때는 `claude mcp remove github`로 연결도 함께 지웁니다.

② Supabase 가입과 MCP 연결

Supabase는 설치 없이 가입만으로 클라우드 데이터베이스를 만들어 주는 플랫폼입니다. Supabase MCP는 노션처럼 원격 MCP라서 토큰 없이 로그인만으로 연결합니다.

STEP 1 가입

`https://supabase.com` → [Sign in] → [Continue with GitHub]로 가입합니다.

STEP 2 조직 생성

'New organization'에서 Type은 'Personal', Plan은 'Free'로 선택하고 [Create organization]을 누릅니다. 조직은 여러 프로젝트를 묶는 폴더와 같습니다.

STEP 3 MCP 추가

VS Code의 터미널 2에서 아래 명령어를 실행합니다.

Supabase MCP 설치 명령어

```
claude mcp add --transport http supabase https://mcp.supabase.com/mcp
```

STEP 4 인증

터미널 1에서 클로드 코드를 다시 실행하고 `/mcp` → supabase 선택 → 'Authenticate'를 누릅니다. 브라우저에서 Supabase 로그인과 접근 승인을 마치면, 2교시 노션 때처럼 연결된 상태로 바뀝니다.

📘 **지금까지 준비된 MCP 서버**

노션(문서 자동화), Playwright(브라우저 조작·자동 테스트), 깃허브(코드 보관), Supabase(데이터베이스) 네 가지입니다. 배포 플랫폼 Vercel은 MCP 없이 깃허브 계정으로 가입만 하면 되므로 오후에 진행합니다.

5교시 · 실습 프로젝트 PART 1 — 앱 제작 → 자동 테스트 → 깃허브 → 배포 13:30 - 15:20

🛠 실습 · 랜딩페이지 리드 수집 앱으로 완성하는 MCP 자동화 개발 & 실서비스

목표: 방문자가 이메일과 관심 분야를 남기면 저장되는 랜딩페이지 리드 수집 웹 앱을 만들고, Playwright MCP로 자동 테스트하고, 깃허브 MCP로 보관하며, Vercel로 배포한 뒤 Supabase MCP로 데이터베이스까지 연결하고 공개 전 보안 설정까지 마쳐 누구나 접속할 수 있는 마케팅 서비스로 완성한다.

STEP 1. 랜딩페이지 리드 수집 앱 만들기

방문자가 이메일과 관심 분야를 제출하면 리드 목록에 추가되고, 각 리드는 체크박스로 '연락 완료'를 표시하거나 삭제할 수 있는 단순한 웹 앱입니다.

'Study-07' 폴더에서 클로드 코드 실행 후 입력

```
이메일과 관심 분야(드롭다운: 마케팅 자동화/콘텐츠 제작/광고 운영/기타)를
입력해 제출하면 아래 목록에 리드가 추가되고, 체크박스를 눌러 '연락 완료'
표시를 하면 글자에 취소선이 그어지며, 삭제 버튼으로 리드를 지울 수 있는
랜딩페이지 리드 수집 웹 앱을 만들어 줘.
HTML/CSS/자바스크립트 한 파일로 lead-capture.html 에 만들어 줘.
```

STEP 1-1 결과물 확인

VS Code 탐색기에 생긴 `lead-capture.html` 을 오른쪽 클릭 → **[Reveal in File Explorer]**(파일 탐색기에 표시)로 폴더를 열고 더블 클릭합니다. 터미널 2에서 `start lead-capture.html` 을 입력해도 브라우저로 열립니다.

STEP 1-2 동작 확인

이메일을 몇 개 제출해 보고, 체크박스와 삭제 버튼이 잘 보이는지 눈으로 확인합니다.

STEP 2. Playwright MCP로 자동 테스트 실행하기

이번에는 직접 클릭하는 대신, 클로드 코드가 브라우저를 조작해 핵심 기능 세 가지(리드 추가, 연락 완료 체크, 삭제)를 차례로 검증하게 합니다.

클로드 코드 프롬프트 입력 예시

```
Playwright MCP를 사용해서 lead-capture.html 을 열고,
리드 추가·연락 완료 체크·삭제 기능이 모두 정상 작동하는지
자동으로 테스트하고 결과를 스크린샷으로 남겨 줘.
```

STEP 2-1 테스트 시작 승인

Playwright MCP로 파일을 열려고 할 때 승인 요청이 뜨면 'Yes'를 눌러 진행합니다.

STEP 2-2 리드 추가 테스트

테스트용 이메일(예: `kim@test.com`, `lee@test.com`, `park@test.com`)을 자동으로 입력하는 요청마다 'Yes'를 누르면, 브라우저가 열리며 리드가 하나씩 추가되는 것을 볼 수 있습니다.

STEP 2-3 연락 완료·삭제 테스트

체크박스를 눌러 취소선이 생기고 해제되는지, 삭제 버튼으로 목록에서 사라지는지 자동으로 점검합니다.

STEP 2-4 스크린샷 저장

테스트가 끝나면 스크린샷 저장을 승인합니다. 'Study-07' 폴더 안 `.playwright-mcp` 폴더에 결과 이미지가 저장됩니다.

> ✋ **여기서 잠깐 · 왜 자동 테스트가 중요할까**
> 사람이 매번 클릭하며 확인하는 테스트는 기능이 늘어날수록 빠뜨리기 쉽습니다. 반복 검증을 AI에게 맡기면 코드를 고칠 때마다 같은 테스트를 몇 초 만에 다시 돌릴 수 있습니다. 캠페인 직전에 리드 수집 폼이 제대로 작동하는지 매번 자동으로 확인할 수 있다는 뜻입니다.

STEP 3. 깃허브 MCP로 프로젝트 보관하기

테스트를 통과한 코드는 안전하게 보관할 차례입니다. 4교시에 연결해 둔 깃허브 MCP를 사용합니다.

> 📘 **용어 정리 · 깃허브 작업 흐름 3단계**
> **추가(add)** : 올릴 파일을 골라 준비 공간(staging area)에 담는 단계입니다.
> **커밋(commit)** : 준비 공간의 파일을 확정해 내 컴퓨터의 로컬 저장소에 기록하는 '저장 단위'입니다.
> **푸시(push)** : 커밋한 내용을 원격 저장소(깃허브)로 올립니다. 이 단계가 끝나야 다른 사람도 변경 내역을 볼 수 있습니다.

'Study-07' 폴더에서 클로드 코드에 입력

```
이 프로젝트를 위한 깃허브 저장소를 새로 만들고,
지금까지 작업한 파일을 모두 커밋해서 저장소에 푸시해 줘.
저장소 이름은 landing-lead-collector 로 해 줘.
```

저장소 생성부터 add·commit·push까지 승인 요청마다 'Yes'를 눌러 끝까지 진행합니다. 알려 주는 저장소 URL로 이동하면 `CLAUDE.md`와 `lead-capture.html`이 올라가 있습니다.

STEP 4. Vercel로 인터넷에 배포하기

배포(deploy)란 내 앱을 인터넷에 연결된 서버에 올려 URL 하나로 어디서든 접속하게 만드는 과정입니다. 랜딩페이지는 광고나 SNS 링크로 들어오는 방문자를 맞이해야 하므로 배포가 필수입니다. Vercel은 깃허브에 올린 코드를 자동으로 배포해 주는 무료 클라우드 플랫폼입니다.

STEP 4-1 Vercel 가입

`https://vercel.com` → [Sign Up] → 'I'm working on personal projects' 선택, 이름 입력 후 [Continue with GitHub]로 가입합니다.

STEP 4-2 새 프로젝트 만들기

[Dashboard] → [Projects] → [Add New...] → [Project]를 클릭합니다.

STEP 4-3 깃허브 저장소 연결

'Import Git Repository'에서 'Add GitHub Account'로 깃허브와 연동하고, 'Only select repositories'에서 'landing-lead-collector'를 골라 [Install]을 누릅니다.

STEP 4-4 불러오기 및 배포

'landing-lead-collector' 옆 [Import] → [Deploy]를 클릭합니다.

> ✋ **여기서 잠깐 · 배포했는데 404 오류가 뜬다면**
> 웹사이트의 기본 시작 페이지는 `index.html` 인데, 우리 파일 이름이 `lead-capture.html` 이라서 생기는 문제입니다. 아래처럼 요청하면 해결됩니다.

```
lead-capture.html 파일 이름을 index.html 로 바꾸고,
변경 내용을 깃허브 저장소에 다시 커밋하고 푸시해 줘.
```

푸시가 끝나면 Vercel이 자동으로 다시 배포합니다. 프로젝트의 'Domains' 항목에 표시된 주소로 접속되는지 확인합니다. **아직 이 주소를 광고나 SNS에 올리지는 않습니다.** 6교시의 보안 설정까지 마친 뒤에 공개합니다.

6교시 · 실습 프로젝트 PART 2 — Supabase 연동과 공개 전 보안 설정 15:30 - 17:10

STEP 5. 로컬 저장소의 한계와 클라우드 데이터베이스

지금 앱에 입력한 리드는 브라우저 안의 '로컬 스토리지'에 저장됩니다. 로컬 스토리지는 지금 이 기기, 이 브라우저에서만 보이므로 동료가 다른 컴퓨터로 접속하면 리드를 볼 수 없고, 캐시를 지우면 데이터가 사라집니다. 여러 기기와 담당자가 같은 데이터를 보려면 4교시에 연결한 Supabase 같은 클라우드 데이터베이스가 필요합니다.

> 📘 **용어 정리 · 데이터베이스와 테이블**
>
> **데이터베이스** : 많은 데이터를 구조화해 저장하고 빠르게 찾거나 고칠 수 있게 해 주는 시스템, 즉 앱의 '기억 저장소'입니다.
>
> **테이블** : 데이터베이스 안에서 데이터를 표 형태로 저장한 구조입니다. 예를 들어 `leads` 테이블의 각 행에는 리드 한 명이, 각 열에는 id·email·interest·status·created_at 같은 속성이 저장됩니다.

STEP 6. 데이터베이스 연동으로 업그레이드하기

클로드 코드 프롬프트 입력 예시

```
Supabase MCP를 사용해서 새 프로젝트와 leads 테이블을 만들고,
index.html 이 로컬 스토리지 대신 이 데이터베이스에서
리드를 읽고 쓰도록 코드를 수정해 줘.
```

STEP 6-1 승인 진행

Supabase MCP 사용 승인과 무료 플랜 새 프로젝트 생성 안내를 확인하고 진행합니다.

STEP 6-2 코드 수정과 테이블 생성

클로드 코드가 `index.html` 을 수정하고, id·email·interest·status·created_at 열을 가진 `leads` 테이블을 만듭니다.

STEP 6-3 대시보드에서 확인

Supabase 대시보드의 새 프로젝트에서 [Table Editor] → [leads]를 열어 열이 실제로 만들어졌는지 확인합니다. 수정한 코드는 깃허브에 푸시해 다시 배포합니다.

STEP 7. 실시간 동기화 확인하기

STEP 7-1 리드 추가

배포된 페이지에서 이메일을 제출하고 [Table Editor]를 새로고침하면 새 행이 추가되어 있습니다.

STEP 7-2 상태 변경·삭제

체크박스로 '연락 완료'를 바꾸면 status 열이, 삭제하면 테이블의 행이 함께 바뀌는지 확인합니다.

STEP 7-3 다른 기기에서 확인

다른 컴퓨터나 시크릿 창으로 같은 주소에 접속해도 같은 목록이 보이면, 데이터가 클라우드에 저장되고 있다는 뜻입니다.

STEP 8. 관심 분야별 리드 현황 — 데이터를 성과 지표로 바꾸기

리드가 쌓이기만 해서는 마케팅에 쓸 수 없습니다. '어떤 관심 분야의 반응이 가장 많았는지'처럼 숫자로 요약해야 다음 캠페인 방향을 정할 수 있습니다. Supabase MCP는 데이터베이스에 직접 질의할 수 있어, 이런 집계를 자연어로 맡길 수 있습니다.

클로드 코드 프롬프트 입력 예시

```
Supabase MCP로 leads 테이블을 조회해서
관심 분야(interest)별 신청 인원과 '연락 완료' 비율을 표로 정리해 줘.
```

원시 데이터를 의사결정에 쓰는 성과 지표로 바꾸는 것이 데이터 마케팅의 핵심입니다. 이 집계는 방문자용 페이지가 아니라 클로드 코드 안에서 담당자만 봅니다.

STEP 9. 공개 전 필수 — 방문자는 '신청'만 할 수 있게 하기

지금 페이지에는 큰 문제가 있습니다. 주소를 아는 **방문자 누구나 다른 사람의 이메일 목록을 보고, 지우고, 상태를 바꿀 수 있습니다.** 실습에서는 동기화를 눈으로 확인하려고 이렇게 만들었지만, 이 상태로 광고에 올리면 개인정보 노출 사고가 됩니다. 방문자 화면과 관리자 화면을 나누고, 데이터베이스의 보안 설정으로 방문자가 할 수 있는 일을 '추가' 하나로 제한합니다.

> 📘 **용어 정리 · RLS(행 수준 보안)**
> Supabase 테이블마다 '누가 어떤 행을 조회·추가·수정·삭제할 수 있는지' 규칙을 정하는 보안 설정입니다. 웹 페이지의 코드는 누구나 볼 수 있으므로, 화면에서 버튼을 숨기는 것만으로는 막을 수 없고 데이터베이스 쪽에서 막아야 합니다.

클로드 코드 프롬프트 입력 예시 ① — 먼저 점검만

```
Supabase MCP 로 leads 테이블의 보안 설정(RLS 정책)을 점검해 줘.
지금 상태에서 방문자가 할 수 있는 일(조회/추가/수정/삭제)을 표로 정리하고,
방문자는 '추가'만 할 수 있게 하려면 무엇을 바꿔야 하는지 설명해 줘.
아직 실제로 바꾸지는 마.
```

클로드 코드 프롬프트 입력 예시 ② — 설명을 읽고 적용

```
방금 설명한 대로 바꿔 줘. index.html 에는 신청 폼과 '신청이 완료되었습니다'
안내만 남기고 리드 목록은 보이지 않게 해. 방문자는 leads 테이블에
추가만 할 수 있도록 RLS 정책을 적용해 줘.
리드 목록은 내가 Supabase 대시보드의 Table Editor 에서 확인할게.
```

STEP 9-1 다시 배포

변경 내용을 깃허브에 커밋·푸시해 달라고 요청하고, Vercel이 다시 배포할 때까지 기다립니다.

STEP 9-2 방문자 입장에서 확인

시크릿 창으로 배포 주소에 접속해 이메일을 제출합니다. 화면에 다른 사람의 이메일이 보이지 않아야 합니다.

STEP 9-3 관리자 입장에서 확인

Supabase [Table Editor]의 `leads`에 방금 제출한 행이 들어왔는지 확인합니다. 리드 현황은 STEP 8처럼 클로드 코드에 물어봅니다.

> ✋ **여기서 잠깐 · 드디어 완성된 진짜 서비스**
> 로컬 파일 하나였던 리드 수집 폼이 자동 테스트를 통과하고, 깃허브에 보관되며, 인터넷 어디서든 접속되고, 데이터가 클라우드에 안전하게 보관되는 서비스가 되었습니다. 실제 캠페인에 쓰기 전에는 개인정보 수집 동의 문구도 필요합니다. 이 부분은 추가 실습 D에서 다룹니다.

부록 · 나만의 북마크 관리 앱 만들기 (선택)

시간이 남는 반은 7교시 전 여유 시간에, 빠듯한 반은 수료 후 개인 과제로 안내합니다.

오늘 연결한 MCP 서버를 그대로 써서, 마케팅과 상관없는 소재로 같은 흐름을 한 번 더 연습합니다. 자주 가는 사이트의 제목·URL·카테고리·메모를 등록하고 조회·삭제하는 앱입니다.

STEP 1 노션 MCP로 후보 리서치

"개발 공부할 때 자주 찾아보게 되는 유용한 사이트를 카테고리(공식 문서/커뮤니티/온라인 도구)별로 5개씩 추천해서 노션에 '내 북마크 후보' 페이지로 정리해 줘."

STEP 2 앱 만들고 Supabase 연결

아래처럼 요청합니다.

```
제목·URL·카테고리·메모를 입력해 등록하면 아래 목록에 추가되고,
각 항목을 삭제 버튼으로 지울 수 있는 북마크 관리 웹 앱을 bookmark.html 한 파일로 만들어 줘.
Supabase MCP로 bookmarks 테이블을 새로 만들어서 로컬 스토리지 대신 이 데이터베이스에서 읽고 쓰도록 해 줘.
```

STEP 3 Playwright MCP로 테스트

"Playwright MCP를 사용해서 bookmark.html을 열고, 북마크 2개를 등록한 뒤 하나를 삭제해서 목록이 정상적으로 갱신되는지 자동으로 테스트해 줘."

STEP 4 깃허브 업로드와 Vercel 배포

"이 프로젝트를 위한 깃허브 저장소를 새로 만들고, bookmark.html 파일 이름을 index.html로 바꿔서 커밋하고 푸시해 줘. 저장소 이름은 my-bookmarks로 해 줘." 이어서 Vercel에서 'my-bookmarks'를 Import해 배포하고, 발급된 URL에서 등록·삭제가 되는지 확인합니다.

> ✋ **여기서 잠깐 · 나만 쓰는 앱이라도**
>
> 배포한 주소는 누구나 열 수 있으므로, 주소를 아는 사람이면 내 북마크를 지울 수 있습니다. 혼자만 쓸 앱이라면 배포하지 않고 내 컴퓨터에서 파일로 열어 쓰거나, STEP 9처럼 보안 설정을 함께 요청합니다. 확장 아이디어로는 카테고리별 필터, 제목·메모 검색, 즐겨찾기 고정 등이 있습니다.

7교시 · 7일 과정 총정리 17:10 - 18:00

7일 과정 한눈에 보기

일차	주제	만든 것과 익힌 것
1	오리엔테이션과 개발 환경 설정	바이브 코딩 개념, 클로드 아티팩트로 자기소개 페이지·마케팅 랜딩페이지 만들어 공유
2	효과적인 프롬프트로 AI 200% 활용하기	명확성·구체성·맥락, 5W1H로 PRD 쓰기, 섹션 단위 구현으로 마케팅 포트폴리오 웹페이지 완성
3	클로드 코드로 첫 프로그램 만들기	클로드 코드 설치, SNS·광고 카피 자동 생성기, 배치 파일 실행, CLAUDE.md와 /init
4	클로드 코드 실전 활용	PRD 기반 단계별 프롬프트, 세션 재개·자동 승인·/compact, 회원관리·매출관리 웹앱 복습 실습
5	게임 제작으로 배우는 체계적인 개발과 관리	할루시네이션과 교차 검증, /memory·/clear, 커스텀 명령어로 퀴즈 게임 유지보수 자동화
6	API로 AI와 공공데이터 연결, 에이전트에게 일 맡기기	API 키와 .env, 포토애드, 서브에이전트(검토·작성), 준비된 PRD로 상권 동일업종 조회 웹
7	MCP로 클로드 코드의 한계 넘어서기	노션·Playwright·깃허브·Supabase MCP, Vercel 배포, RLS로 공개 전 보안 설정

일곱 날의 실습은 소재가 달라도 같은 흐름을 따랐습니다. **무엇을 만들지 문서(PRD)로 정하고 → 작게 나눠 요청하고 → 결과를 직접 확인하고 → 고치고 보관한다.** 수료 후 새 도구를 만들 때도 이 순서를 그대로 씁니다.

오늘의 정리

핵심 키워드 MCP · 컨텍스트·컨텍스트 창 · 클라이언트-서버 모델 · 로컬 MCP·원격 MCP · 인증 · Playwright MCP · 토큰 · add·commit·push · 배포(Vercel) · 데이터베이스(Supabase) · 테이블 · RLS(행 수준 보안) · 리드 현황(성과 지표)

- MCP는 AI 모델이 외부 데이터·도구·서비스와 연결될 때 쓰는 표준 통신 규약으로, 여러 API·파일·데이터베이스를 통합해 AI가 직접 접근하고 조작할 수 있게 해 줍니다.

- 컨텍스트는 AI가 지금 보고 판단할 수 있는 정보 전체(책상 위의 서류)입니다. MCP는 바깥 서비스의 정보를 이 책상 위로 가져오는 표준 창구이고, 책상이 어지러워지면 `/clear` · `/compact`로 정리하며, 현재 상태는 `/context`로 확인합니다.
- 원격 MCP(노션·깃허브·Supabase)는 주소와 로그인만으로 연결하고, 로컬 MCP(Playwright)는 Node.js로 내 컴퓨터에서 실행합니다. 출처가 확실한 MCP만 설치합니다.
- Playwright MCP는 브라우저를 직접 조작해 웹 앱 기능을 자동으로 테스트합니다.
- 깃허브 업로드는 add → commit → push 순서이며, 토큰은 필요한 권한만 짧은 기간으로 발급합니다. Vercel은 깃허브의 코드를 자동 배포하고, Supabase는 설치 없이 클라우드 데이터베이스를 제공합니다.
- 데이터베이스에 연결하면 여러 기기·여러 사용자가 같은 데이터를 공유합니다. 개인정보를 모으는 페이지는 공개 전에 RLS로 방문자가 할 수 있는 일을 제한합니다.

확인 문제

Q1. MCP와 API의 차이점으로 가장 적절한 설명을 고르세요.

① API는 특정 기능을 요청하는 방식이고, MCP는 AI가 외부 시스템과 직접 소통하게 하는 프로토콜이다.
② MCP는 API보다 속도가 항상 빠르다.
③ API는 유료이고 MCP는 항상 무료이다.
④ MCP는 클로드 코드에서만 사용할 수 있다.

Q2. 이메일을 모으는 리드 수집 페이지를 광고에 공개하기 전에 반드시 해야 할 일로 가장 알맞은 것을 고르세요.

① 페이지 디자인을 화려하게 바꾼다.
② 방문자는 신청(추가)만 할 수 있도록 데이터베이스 보안 설정(RLS)을 적용하고, 리드 목록은 관리자만 보게 한다.
③ 깃허브 토큰의 유효 기간을 최대한 길게 늘린다.
④ 리드 목록을 페이지 맨 위에 크게 보여 준다.

Q3. 깃허브(GitHub)에 코드를 업로드하는 올바른 순서를 고르세요.

① commit → add → push ② push → add → commit ③ add → commit → push ④ add → push → commit

Q4. 로컬 스토리지와 비교했을 때, 클라우드 데이터베이스(Supabase 등)를 연결하면 무엇이 가장 크게 달라지는지 서술하세요.

→ ( )

Q5. 원격 MCP와 로컬 MCP에 대한 설명으로 옳지 않은 것을 고르세요.

① 노션 MCP는 원격 MCP라서 주소와 로그인(인증)만으로 연결했다.

② Playwright MCP는 로컬 MCP라서 내 컴퓨터에 Node.js가 필요했다.

③ MCP 서버는 누구나 만들 수 있으므로 출처를 확인하지 않고 설치해도 안전하다.

④ `claude mcp list`로 설치된 서버와 연결 상태를 확인할 수 있다.

> 🎓 **수료를 축하합니다**
>
> 7일 동안 정말 고생 많으셨습니다. 코딩을 전혀 몰랐던 1일 차의 여러분은 아티팩트로 첫 웹페이지를 만들었고, 프롬프트를 다듬어 원하는 결과를 끌어내는 법을 익혔으며, 클로드 코드를 설치해 PRD 기반으로 단계별 앱을 완성하고, 게임과 API 연동 프로젝트, 공공데이터 상권 조회 웹을 거치며 서브에이전트에게 일을 나눠 맡겨 봤습니다. 그리고 오늘, MCP로 노션·깃허브·데이터베이스를 연결해 데이터가 클라우드에 안전하게 저장되는 진짜 서비스를 완성하며 7일간의 여정을 마무리했습니다.
>
> 이제부터는 정해진 커리큘럼이 아니라 여러분 자신의 아이디어가 다음 과제입니다. 사소한 불편을 해결하는 작은 도구부터 시작해 보시고, 막히면 클로드 코드 공식 문서와 커뮤니티, 그리고 오늘처럼 클로드에게 직접 물어보며 답을 찾아가시기 바랍니다. 수료를 진심으로 축하드립니다.

추가 실습 · 7일 차 보강 실습 모음

본문 시간표가 예상보다 일찍 끝나는 교시에 넣을 수 있는 실습 네 가지입니다. 7일 차는 설치와 가입이 많아 반마다 진행 속도 차이가 큽니다. 빨리 끝난 수강생에게 개별 과제로 주거나, 반 전체가 여유 있을 때 함께 진행합니다.

실습	넣을 위치	소요 시간
A. 노션에 콘텐츠 캘린더 만들고 읽어 오기	2교시 리서치 노트 실습 뒤	25분
B. MCP 서버 점검·삭제·재설치	4교시 끝	20분
C. 배포 전 점검표로 자동 테스트하기	5교시 STEP 2 뒤	30분
D. 리드 수집 페이지 운영 준비	6교시 STEP 9 뒤	35분

추가 실습 A. 노션에 콘텐츠 캘린더 만들고 읽어 오기

넣을 위치: 2교시 리서치 노트 실습 뒤 · 소요 시간: 25분 · 준비물: 인증을 마친 노션 MCP

본문에서는 노션에 문서를 '쓰는' 것만 해 봤습니다. MCP는 양방향이므로 노션의 내용을 '읽어서' 작업에 쓰는 것까지 확인합니다.

STEP 1 노션에 표(데이터베이스) 만들기

클로드 코드 프롬프트 입력 예시

```
노션에 '콘텐츠 캘린더'라는 데이터베이스를 새로 만들어 줘.
속성은 제목, 채널(인스타그램/블로그/이메일 중 선택),
발행일(날짜), 상태(기획중/작성중/발행완료 중 선택)로 하고,
이번 달 날짜로 예시 콘텐츠 6개를 채워 줘.
```

STEP 2 노션에서 확인하고 직접 고치기

노션에 접속해 표가 만들어졌는지 확인합니다. 그중 한 행의 상태를 손으로 '발행완료'로 바꾸고, 새 행을 하나 직접 추가합니다.

STEP 3 클로드 코드가 읽어 오게 하기

클로드 코드 프롬프트 입력 예시

```
노션의 '콘텐츠 캘린더'를 읽어서, 아직 발행되지 않은 콘텐츠를
발행일이 빠른 순서로 정리해 줘. 채널별로 몇 개씩 남았는지도 알려 줘.
```

STEP 2에서 손으로 바꾼 내용(발행완료 처리한 행, 직접 추가한 행)이 답에 반영됐는지 확인합니다.

STEP 4 읽은 내용으로 작업시키기

클로드 코드 프롬프트 입력 예시

```
발행일이 가장 빠른 콘텐츠 하나를 골라 초안을 써서
그 행의 페이지 본문에 넣고, 상태를 '작성중'으로 바꿔 줘.
```

> 📘 **이 실습에서 확인한 것**
> 사람이 노션에서 고친 내용을 클로드 코드가 읽고, 클로드 코드가 쓴 내용을 사람이 노션에서 봅니다. 파일을 주고받지 않아도 같은 문서를 사이에 두고 함께 일하는 구조이며, 팀이 이미 쓰는 도구에 AI를 연결한다는 MCP의 쓰임새가 여기에 있습니다.

추가 실습 B. MCP 서버 점검·삭제·재설치

넣을 위치: 4교시 끝 · 소요 시간: 20분 · 준비물: 오전에 설치한 MCP 서버 4개

오전에 설치만 네 번 했습니다. 실제로 쓰다 보면 연결이 끊기거나 필요 없어진 서버를 정리해야 하므로, 관리 명령어를 한 번씩 써 봅니다. VS Code의 터미널 2(클로드 코드가 실행되지 않은 터미널)에서 진행합니다.

MCP 관리 명령어

```
claude mcp list                    설치된 서버 목록과 연결 상태 보기
claude mcp get playwright          서버 하나의 설정 자세히 보기
claude mcp remove playwright       서버 삭제하기
```

STEP 1 목록과 상태 확인

`claude mcp list`를 실행해 네 서버가 모두 연결된 상태로 표시되는지 확인합니다.

STEP 2 설정 들여다보기

`claude mcp get playwright`를 실행합니다. 3교시에 입력한 실행 명령(`cmd /c npx …`)과 범위(local)가 그대로 저장되어 있는지 봅니다.

STEP 3 삭제하고 확인하기

`claude mcp remove playwright`를 실행한 뒤 다시 `claude mcp list`로 목록에서 사라졌는지 확인합니다.

STEP 4 다시 설치하기

3교시의 설치 명령어를 다시 실행하고, `claude`를 켜서 `/mcp`로 연결된 상태인지 확인합니다.

STEP 5 다른 폴더에서 확인하기

터미널 2에서 `cd ..\Study-06`으로 'Study-06' 폴더로 이동해 `claude mcp list`를 실행합니다. 오늘 설치한 서버가 보이지 않는 것을 확인합니다. 범위가 local이라 'Study-07' 프로젝트에만 적용되었기 때문입니다. 확인 후 `cd ..\Study-07`로 돌아옵니다.

증상	점검 순서
목록에 실패로 표시됨	① VS Code를 완전히 닫고 다시 열어 실행 ② `node -v`로 Node.js 확인 ③ 삭제 후 재설치
원격 MCP가 인증 필요로 표시됨	`/mcp`에서 해당 서버를 선택해 다시 인증
다른 프로젝트에서 서버가 안 보임	범위가 local인지 확인. 모든 프로젝트에서 쓰려면 `-s user`로 다시 설치

추가 실습 C. 배포 전 점검표로 자동 테스트하기

넣을 위치: 5교시 STEP 2 뒤 · 소요 시간: 30분 · 준비물: lead-capture.html, Playwright MCP

본문의 자동 테스트는 정상적인 입력만 확인합니다. 실제 방문자는 잘못된 값을 넣고, 대부분 휴대폰으로 접속합니다. 점검표를 문서로 만들어 두고 그대로 테스트하게 합니다.

STEP 1 점검표 문서 만들기

클로드 코드 프롬프트 입력 예시

```
lead-capture.html 을 배포하기 전에 확인할 점검표를 test-checklist.md 로
만들어 줘. 아래 항목을 포함하고, 항목마다 기대 결과를 적어 줘.
1) 이메일을 비우고 제출  2) '@'가 없는 이메일로 제출
3) 같은 이메일을 두 번 제출  4) 관심 분야를 고르지 않고 제출
5) 화면 너비 375px(휴대폰)에서 버튼과 입력창이 잘리지 않는지
```

STEP 2 점검표대로 테스트 실행

클로드 코드 프롬프트 입력 예시

```
Playwright MCP 로 test-checklist.md 의 항목을 순서대로 테스트하고,
항목별 통과/실패를 같은 파일에 기록해 줘.
휴대폰 화면 크기에서의 모습은 스크린샷으로 남겨 줘.
실패한 항목이 있어도 아직 코드는 고치지 마.
```

STEP 3 실패 항목 고치고 다시 테스트

기록된 결과를 읽고 실패한 항목을 확인합니다. "실패한 항목을 고치고, 같은 점검표로 처음부터 다시 테스트해 줘"라고 요청해 모두 통과로 바뀌는지 확인합니다.

✋ **여기서 잠깐 · 점검표를 파일로 남기는 이유**

이후 기능을 추가하거나 문구를 바꿀 때마다 "test-checklist.md대로 다시 테스트해 줘" 한 줄이면 같은 검증을 반복할 수 있습니다. 캠페인 시작 직전에 폼이 작동하는지 확인하는 절차를 사람의 기억이 아니라 문서에 맡기는 것입니다.

추가 실습 D. 리드 수집 페이지 운영 준비

넣을 위치: 6교시 STEP 9 뒤 · 소요 시간: 35분 · 준비물: Supabase까지 연결해 배포한 페이지

본문 STEP 9에서 방문자가 다른 사람의 정보를 볼 수 없도록 막았습니다. 실제 캠페인에 쓰려면 하나가 더 필요합니다. 이메일은 개인정보이므로 수집 전에 동의를 받아야 합니다.

STEP 1 수집 동의와 중복 방지 추가

클로드 코드 프롬프트 입력 예시 ①

```
index.html 의 폼에 '개인정보 수집·이용에 동의합니다' 체크박스를 추가하고,
체크하지 않으면 제출되지 않게 해 줘. 수집 항목(이메일, 관심 분야),
이용 목적, 보유 기간을 적는 안내문 자리도 만들어 줘.
같은 이메일이 두 번 저장되지 않게 데이터베이스에서 막고, 이미 있으면
'이미 신청하셨습니다'라고 안내해 줘. 방문자는 추가만 할 수 있는
지금의 보안 설정(RLS)은 그대로 유지해 줘.
```

STEP 2 리드를 엑셀로 가져가기

Supabase [Table Editor]에서 leads 테이블을 열고 내보내기(Export) 기능으로 CSV를 내려받아 엑셀로 열어 봅니다. 또는 클로드 코드에 "Supabase MCP로 leads 테이블을 조회해서 관심 분야별 인원과 최근 7일 신청 수를 정리해 줘"라고 요청합니다.

운영 전 확인 항목	내용
수집 동의	동의 체크 없이는 제출되지 않는가. 안내문의 항목·목적·보유 기간을 회사 기준으로 채웠는가
접근 권한	방문자 화면에서 다른 사람의 정보가 보이거나 지워지지 않는가
토큰 관리	깃허브 토큰이 코드나 저장소에 올라가 있지 않은가. 과정이 끝난 뒤 토큰을 삭제했는가
무료 플랜 한계	Supabase 무료 프로젝트는 오래 사용하지 않으면 일시 중지될 수 있으므로 캠페인 전에 접속해 확인

✋ **여기서 잠깐 · 실제 캠페인에 쓰기 전에**

개인정보 수집 안내문의 문구와 보유 기간은 회사의 개인정보 처리방침에 맞춰야 하므로, 클로드 코드가 써 준 문구를 그대로 쓰지 말고 법무·개인정보 담당 부서의 확인을 받습니다. 이 실습은 무엇을 점검해야 하는지 익히는 것이 목적입니다.